SOFTWARE REQUIREMENTS SPECIFICATION

For

Traffic Guardian Pro

AI-Powered Dashcam Violation Detection System

Date: 14 March 2026

Specialization	SAP ID	Name
Computer Science (CCVT)	500125838	Rishi Yadav
Computer Science (CCVT)	500119639	Gaurav Singh
Computer Science (CCVT)	500125831	Abhay Agnihotri
Computer Science (CCVT)	500121139	Sanyam Dixit

CCVT Cluster

School of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES,

DEHRADUN–248007. Uttarakhand

TABLE OF CONTENTS

B	Appendix B: Analysis Model	
C	Appendix C: Issues List	

REVISION HISTORY

Date	Change	Reason for Changes	Mentor Signature
14 Mar 2026	v1.0 Initial Draft	Project Ssubmission; Minor 1	

1 INTRODUCTION

1.1 Purpose of the Project

Traffic Guardian Pro is a real-time AI-powered dashcam application designed to automatically detect wrong-way vehicles from a moving vehicle's camera feed, capture photographic evidence of the violation, read the offending vehicle's license plate using Optical Character Recognition (OCR), and log all events to a persistent, timestamped database. The core motivation for this project is the significant contribution of wrong-way driving incidents to road fatalities on Indian highways. Existing traffic surveillance systems are primarily stationary CCTV installations, which have limited coverage and require manual monitoring. This project demonstrates that consumer-grade hardware (a laptop and webcam/dashcam) combined with state-of-the-art lightweight deep learning models can serve as an effective, mobile enforcement tool, enabling citizen-led violation recording without any dependency on fixed infrastructure.

1.2 Target Beneficiary

- Traffic police and highway patrol units deploying the system in patrol vehicles.
- State and national highway authorities seeking cost-effective surveillance solutions.
- Smart city infrastructure projects integrating AI-driven traffic management.
- Citizens and civic bodies seeking to report and document wrong-way driving incidents.
- Researchers and students in the domain of computer vision and intelligent transportation systems.

1.3 Project Scope

The system processes a live 1280×720 pixel video feed from a dashcam or webcam mounted on a moving vehicle. It detects and tracks vehicles using the YOLOv8n object detection model (pretrained on the COCO dataset, classes: car, motorbike, bus, truck). Wrong-way vehicles are identified using a multi-factor scoring engine that evaluates three independent signals: vertical approach speed, bounding-box growth rate, and lane position. A vehicle is flagged only when its violation score reaches or exceeds 5 out of 7 across a minimum of four consecutive detection frames, significantly reducing false positives versus single-factor detection approaches.

Upon confirmed violation detection, the system triggers a point-blank capture: a high-resolution crop of the vehicle is processed through an OCR pipeline (plate-strip isolation, 3× upscaling, bilateral filtering, CLAHE contrast enhancement, dual-threshold binarisation, EasyOCR inference, Indian plate regex validation) to extract the license plate number. All events are written to a CSV database using a threading lock, ensuring data integrity under concurrent captures. The system provides a live Tkinter dashboard with a real-time video feed, event log, evidence preview, and session statistics.

The project does not include integration with any government database (e.g., VAHAN), network transmission of evidence, or prosecution/enforcement functionality – these are identified as future scope items.

1.4 References

- Redmon, J. & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767.
- Jocher, G. et al. (2023). Ultralytics YOLOv8. Available at: https://github.com/ultralytics/ultralytics
- JaidedAI (2020). EasyOCR. Available at: https://github.com/JaidedAI/EasyOCR
- Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
- Indian Motor Vehicles Act, 1988 – Section 184: Driving dangerously (wrong-way provisions).
- IS/ISO 39001:2012 – Road Traffic Safety Management Systems.

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

The core algorithmic contribution of this project is the Multi-Factor Wrong-Way Vehicle Scoring Engine, designed to replace the naive single-threshold approach (delta_y > 50) that produced a high false positive rate. The algorithm computes a score out of 7 per tracked vehicle per frame as follows:

Factor	Condition	Points Awarded
Vertical approach speed (delta_y over 8 frames)	delta_y > 35 px	**+3**
Vertical approach speed (delta_y over 8 frames)	delta_y 20–35 px	**+1**
Bounding-box width growth rate	growth > 15%	**+3**
Bounding-box width growth rate	growth 8–15%	**+1**
Lane position (centre-x)	cx in right 65% of frame	**+1**

Capture is triggered when: (i) score ≥ 5/7, (ii) violation persists for ≥ 4 consecutive frames (anti-jitter), (iii) vehicle bottom edge crosses the capture zone (70% of frame height), (iv) bounding-box width > 120px, (v) vehicle not already captured in the current session.

The OCR pipeline applies the following ordered pre-processing steps to maximise plate readability: (1) Plate-strip crop – bottom 40% of bounding box. (2) 3× bicubic upscale. (3) Bilateral filter (d=11, sigmaColor=17, sigmaSpace=17). (4) CLAHE (clipLimit=3.0, tileGridSize=8×8). (5) Dual threshold – Otsu and Adaptive Gaussian. (6) EasyOCR inference on both; select result with higher average confidence (≥ 0.4 per token). (7) Indian plate regex validation: [A-Z]{2}[0-9]{1,2}[A-Z]{1,3}[0-9]{4}.

2.2 Data / Data Structure

Input data is a live video stream from a USB webcam or dashcam device at 1280×720 resolution and up to 30 FPS, captured via OpenCV VideoCapture. No custom training dataset is required; YOLOv8n uses weights pretrained on the COCO benchmark dataset (80 classes; relevant classes: 2=car, 3=motorbike, 5=bus, 7=truck).

Vehicle tracking state is maintained in memory using two Python dictionaries: vehicle_history (maps object ID to a list of (cx, bottom_y, box_width) tuples – 12-frame rolling buffer) and violation_counter (maps object ID to consecutive violation frame count). Evidence images are saved as JPEG files with microsecond-precision timestamps to prevent filename collisions. The persistent violation log is stored as a CSV file (traffic_data.csv) with columns: Timestamp, Violation, Plate, Evidence_File, Total_Offenses.

Category	Count	Percentage (%)
Total Observations	198	100%
Wrong Way Detection	109	55.05%
Correct Movement	88	44.44%
False Detection	7	3.54%
Night Violations	2	1.01%
Illegal Crossing	—	—

2.3 SWOT Analysis

STRENGTHS	WEAKNESSES
+ No fixed infrastructure required + Works from any moving vehicle + Real-time detection with visual evidence + Thread-safe persistent CSV database + Indian plate regex validation reduces noise	- Accuracy limited by camera quality - OCR struggles with partial/obscured plates - Night-time requires IR camera - CPU-only; limited to ~10 FPS on laptops
OPPORTUNITIES	**THREATS**
+ VAHAN DB API for instant owner lookup + Cloud upload for enforcement portal + Mobile deployment via TFLite export + Multi-violation-type expansion (speed, signal)	- Privacy regulations on plate recording - Adversarial plates (mud, foreign format) - Rapid camera motion causing missed frames - Windshield reflections degrading OCR

2.4 Project Features

- Real-time vehicle detection at ~10 FPS using YOLOv8n with frame-skipping optimization.
- Spatial ID tracking with 12-frame history buffer for stable object identity across camera motion.
- Multi-factor wrong-way scoring engine (7-point scale) with anti-jitter 4-frame confirmation.
- Point-blank capture zone: violation evidence captured only when vehicle is large and close.
- OCR pipeline with plate-strip isolation, 3× upscaling, dual-threshold, and confidence filtering.
- Indian license plate regex validation to reject garbage OCR reads.
- Thread-safe CSV database with threading.Lock preventing concurrent write corruption.
- Microsecond-precision evidence image filenames preventing file overwrite collisions.
- Repeat offender detection with incremental offense counter and elevated audio alert.
- Live Tkinter dashboard: real-time annotated video feed, event log, evidence preview, session statistics.
- Visual capture-zone overlay showing active detection boundary on the video feed.

2.5 User Classes and Characteristics

Primary User – Vehicle Operator / Traffic Officer: Interacts with the Tkinter dashboard to start/stop monitoring. Requires no technical knowledge; interaction is limited to two buttons (Start/Stop). The system operates autonomously once started.

Secondary User – Data Analyst / Enforcement Officer: Reviews the generated CSV database and evidence images to process violations. Requires basic computer literacy to open CSV files and view JPEG evidence images.

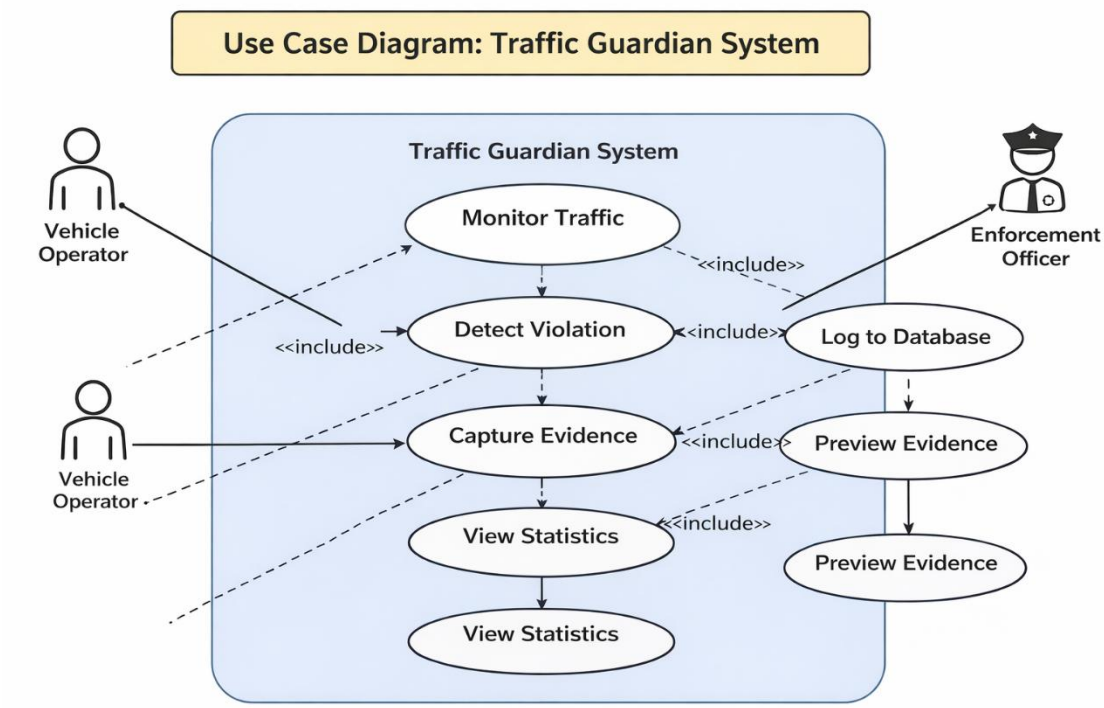
System Administrator (optional): Manages the software installation, model file placement (yolov8n.pt), and hardware configuration (camera index, resolution). Requires Python 3.9+ environment setup knowledge.

2.6 Design and Implementation Constraints

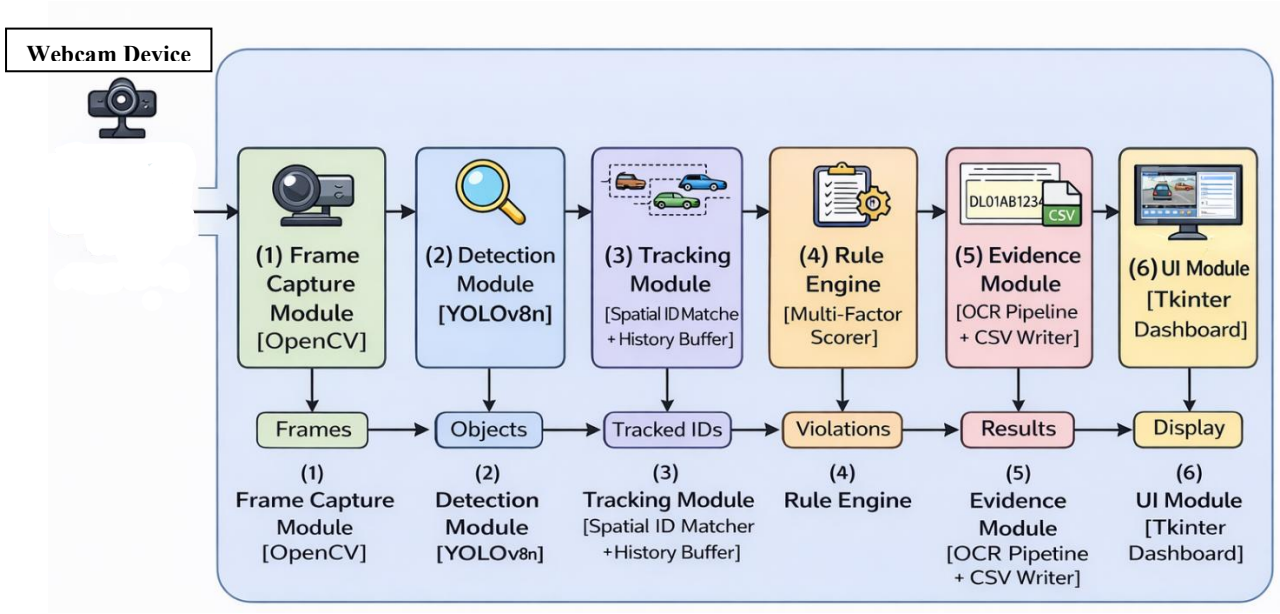
- Hardware: Minimum CPU with 4 cores at 2.5GHz for real-time frame processing. No GPU required (EasyOCR configured with gpu=False). Minimum 8GB RAM recommended for YOLOv8n + EasyOCR concurrent operation.
- Operating System: Windows (winsound module used for audio alerts). Porting to Linux/macOS requires replacing winsound with an alternative audio library.
- Camera: Minimum 720p (1280×720) resolution dashcam or USB webcam. IR capability required for night-time operation (not included in current scope).
- Software: Python 3.9+, ultralytics, easyocr, opencv-python, Pillow, tkinter (standard library), threading (standard library).
- Capture Zone: Hardcoded at 70% of frame height (configurable via zone_y variable). Must be recalibrated for different vehicle mounting heights.
- Language: License plate validation regex is specific to Indian plate format. International deployment requires regex modification.
- Performance: Frame processing limited to every 3rd frame (skip_frames=3) to maintain UI responsiveness on CPU-only hardware.

2.7 Design Diagrams

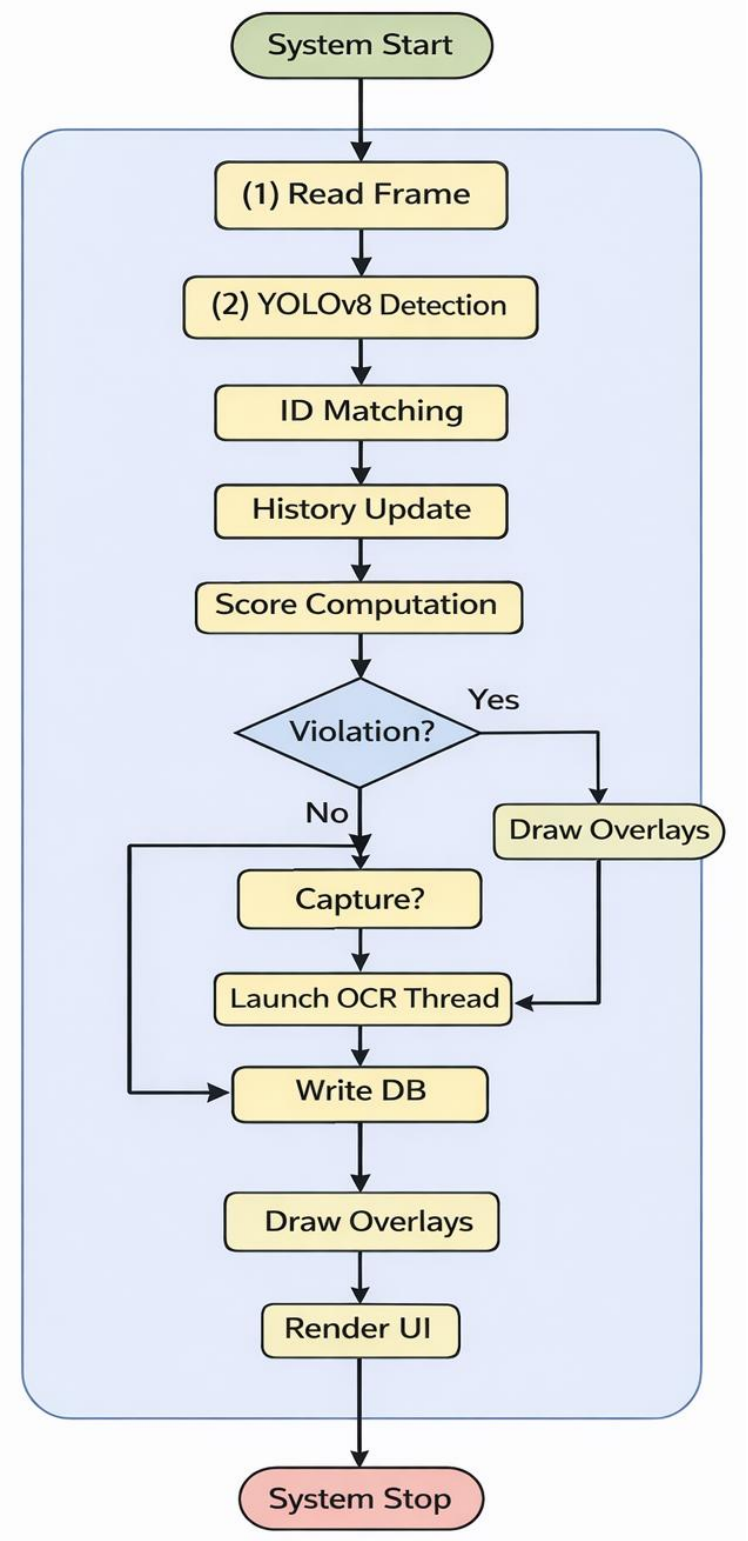
Use Case Diagram: Two primary actors: Vehicle Operator (starts/stops system, views dashboard) and Enforcement Officer (reviews CSV + evidence images). Main use cases: Monitor Traffic, Detect Violation, Capture Evidence, Log to Database, Preview Evidence, View Statistics.



System Data Flow: (1) Frame Capture Module [OpenCV] → (2) Detection Module [YOLOv8n] → (3) Tracking Module [Spatial ID Matcher + History Buffer] → (4) Rule Engine [Multi-Factor Scorer] → (5) Evidence Module [OCR Pipeline + CSV Writer] → (6) UI Module [Tkinter Dashboard].



Activity Diagram: System Start → Continuous Frame Loop: [Read Frame → YOLO Detection → ID Matching → History Update → Score Computation → Violation? (Yes → Capture? → Launch OCR Thread → Write DB) → Draw Overlays → Render UI] → System Stop.



2.8 Assumptions and Dependencies

- Assumption: The dashcam is mounted behind the windshield facing forward, providing a clear frontal view of oncoming vehicles.
- Assumption: Oncoming (wrong-way) vehicles travel in the right portion of the camera frame (consistent with Indian left-hand drive roads).
- Assumption: The capture zone threshold (70% frame height) is appropriate for typical dashcam mounting heights. Re-calibration required for non-standard mounts.
- Dependency: ultralytics YOLOv8n pretrained weights file (yolov8n.pt) must be present in the working directory or auto-downloaded on first run.
- Dependency: EasyOCR language model files are downloaded automatically on first initialisation (English model).
- Dependency: OpenCV compatible camera device must be connected and accessible at camera_index=0.

3 SYSTEM REQUIREMENTS

3.1 User Interface

The system provides a desktop GUI built with Python's Tkinter library. The interface consists of the following components:

- Header Bar: Application title ("Traffic Guardian"), edition label, institutional branding.
- Main Video Panel: Full-resolution annotated live video feed (960×540 display) with bounding boxes (green for safe, red for wrong-way), violation labels with score, and the red capture-zone line.
- Status Card: Online/Offline indicator with colour coding (green/red).
- Statistics Card: Live violation counter (large numeric display).
- Controls Card: Start Monitoring button and Stop System button with state-appropriate enabling/disabling.
- Evidence Preview Pane: Displays the most recently captured vehicle evidence image (360×200 pixels).
- Live Event Log: Scrollable console-style text area showing timestamped events with colour-coded alerts (red for repeat offenders).

3.2 Software Interface

The system integrates the following external software components:

- YOLOv8n (Ultralytics): Object detection inference. Input: BGR numpy array (1280×720). Output: Bounding box coordinates, class IDs, confidence scores.
- EasyOCR (JaidedAI): Text recognition. Input: Pre-processed grayscale numpy array. Output: List of (bbox, text, confidence) tuples.
- OpenCV (cv2): Camera capture, image processing (resize, bilateral filter, CLAHE, threshold, resize), bounding box drawing, JPEG file writing.
- Pillow (PIL): BGR-to-RGB conversion, Tkinter-compatible PhotoImage generation for UI rendering.
- Python threading: Background OCR thread execution to prevent UI blocking during plate recognition.

3.3 Database Interface

The system uses a flat-file CSV database (traffic_data.csv) stored in the application's working directory. The schema is as follows:

Column	Type	Format	Description	Example
Timestamp	String	YYYY-MM-DD HH:MM:SS	Event date and time	2026-03-14 11:30:45
Violation	String	Text	Violation type	Wrong Way

Plate	String	Alphanumeric	Extracted plate number	MH12AB1234
Evidence_File	String	Filename.jpg	Path to evidence image	evidence_20260314_113045_123456.jpg
Total_Offenses	Integer	Numeric	Cumulative offense count for this plate	3

Concurrent write safety is ensured via a threading.Lock (db_lock) that serialises all read-modify-write operations on the CSV file. Each event always appends a new row; repeat-offender rows additionally update the existing plate record's Total_Offenses counter.

3.4 Protocols

The current system is entirely offline; no network communication protocols are employed. All data (evidence images and CSV records) are stored locally on the host machine. Future versions may implement HTTPS REST API communication to upload evidence to a centralised enforcement portal. No encryption or authentication mechanisms are implemented in the current scope. Inter-thread communication uses Python's threading primitives (Lock, Thread) and Tkinter's after() callback mechanism for thread-safe UI updates.

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

- Detection Latency: YOLO inference executes on every 3rd frame, targeting a UI update rate of approximately 30 FPS (33ms per frame). On a 4-core 2.5GHz CPU, effective detection processing rate is ~10 FPS.
- OCR Throughput: EasyOCR processing per capture executes asynchronously in a background thread; typical processing time is 0.5–2 seconds depending on image complexity. The is_ocr_busy flag prevents overlapping concurrent OCR calls.
- Memory: YOLOv8n model occupies ~12MB. EasyOCR English model ~40MB. Combined runtime RAM usage: approximately 500MB–1.2GB.
- Storage: Each evidence image (JPEG, quality default) is approximately 50–150KB. A full 8-hour shift recording at one violation per 5 minutes produces approximately 100 records and 10–15MB of evidence images.
- Anti-Jitter: 4-frame confirmation window prevents single-frame noise from triggering false captures.

4.2 Security Requirements

- Data Storage: All violation records and evidence images are stored locally; no data is transmitted over any network in the current version.
- Access Control: The application does not implement user authentication. Physical access control to the host machine is assumed.
- Privacy: License plate images constitute personally identifiable information (PII) under applicable data protection regulations. Deployment must comply with relevant state/national privacy laws.
- File Integrity: CSV file is protected against concurrent corruption via threading.Lock. No encryption of stored data is implemented in the current scope.

4.3 Software Quality Attributes

- Reliability: Thread-safe database operations with explicit locking ensure no data loss under concurrent detection events. Anti-jitter logic and multi-factor scoring reduce unreliable false detections.
- Maintainability: The system is structured in clearly separated modules (detection, tracking, scoring, OCR, database, UI). Violation scoring parameters (threshold, factor weights) are defined as constants for easy recalibration.
- Portability: Core logic (YOLOv8, OpenCV, EasyOCR) is cross-platform. The winsound dependency limits the current build to Windows; replacement with playsound or simpleaudio achieves cross-platform audio.
- Usability: The Tkinter dashboard provides a single-window interface with two-button operation (Start/Stop). No technical knowledge is required for routine operation.

- Extensibility: The scoring engine and OCR pipeline are modular; additional violation types (e.g., signal jumping, speeding via optical flow) can be added as new scoring factors without altering the core tracking architecture.
- Testability: The multi-factor scorer is a pure function with deterministic outputs, enabling unit testing with controlled input vectors without requiring a live camera feed.

5 OTHER REQUIREMENTS

Installation Requirement: The host machine must have Python 3.9 or later installed with the following packages: ultralytics, easyocr, opencv-python, Pillow. The YOLOv8n weights file (yolov8n.pt) is downloaded automatically on first run if not present. EasyOCR English language model files are downloaded automatically on first initialisation.

Camera Calibration Requirement: The capture zone threshold (zone_y = 70% of frame height) and bounding-box size threshold (box_width > 120px) may require adjustment based on dashcam mounting position, focal length, and typical traffic density on the deployment road segment.

Regulatory Compliance: Users must obtain appropriate authorisation from the relevant traffic authority before using recorded evidence for official enforcement purposes. The system is intended as a supplementary tool, not a replacement for legally mandated enforcement procedures.

APPENDIX A: GLOSSARY

Term	Definition
YOLO	You Only Look Once – a family of real-time object detection neural networks.
YOLOv8n	Nano variant of YOLOv8; smallest and fastest model in the Ultralytics YOLOv8 family.
OCR	Optical Character Recognition – automated extraction of text from images.
EasyOCR	Open-source multi-language OCR library by JaidedAI.
COCO	Common Objects in Context – large-scale object detection benchmark dataset.
CLAHE	Contrast Limited Adaptive Histogram Equalization – local contrast enhancement technique.
Bilateral Filter	Edge-preserving image smoothing filter that reduces noise while maintaining sharp edges.
Bounding Box (BBox)	Rectangular region returned by YOLO defining the spatial extent of a detected object.
delta_y	Change in the bottom edge y-coordinate of a vehicle bounding box over 8 frames.
Multi-factor Scoring	Violation detection approach combining multiple independent evidence signals into a weighted score.
threading.Lock	Python synchronisation primitive that prevents concurrent access to a shared resource.
VAHAN	Vehicle Registration & Licensing Information System – MoRTH national vehicle database.
LHD	Left-Hand Drive – traffic system where vehicles drive on the left side of the road (India).
FPS	Frames Per Second – rate at which video frames are captured or processed.
CSV	Comma-Separated Values – plain-text tabular data format used for the violation log.

APPENDIX B: ANALYSIS MODEL

The primary analysis model for this project is the Multi-Factor Wrong-Way Detection Scoring Engine (described in Section 2.1). The model was derived by analysing failure modes of the original single-factor approach and identifying three orthogonal signals that together distinguish genuine wrong-way oncoming vehicles from false positive scenarios:

Factor 1 – Vertical Approach Speed (max 3 points): A vehicle approaching head-on at speed will generate a large positive delta_y (bottom edge moving rapidly downward in frame). A parked car being passed or slow same-direction traffic generates low or near-zero delta_y. This factor is the strongest discriminator and receives the highest weight.

Factor 2 – Bounding-Box Width Growth Rate (max 3 points): A head-on vehicle rapidly expands in apparent size as it approaches. A same-direction vehicle or stationary object does not show significant width growth. Combined with Factor 1, this eliminates false positives from vehicles in adjacent lanes that may have high vertical displacement due to lane geometry.

Factor 3 – Lane Position (max 1 point): In left-hand drive countries (India), oncoming vehicles occupy the right portion of the camera frame. This bonus point biases the score toward geometrically consistent wrong-way scenarios without being a hard requirement, allowing the system to detect edge cases (e.g., head-on collision scenarios on single-lane roads).

APPENDIX C: ISSUES LIST

#	Issue	Status	Priority	Resolution / Notes
1	False violations on same-direction traffic	**RESOLVED**	HIGH	Multi-factor scoring engine (v2) replaces single delta_y check.
2	CSV records not written (thread collision)	**RESOLVED**	CRITICAL	threading.Lock serialises all DB writes.
3	Evidence filename collision (same second)	**RESOLVED**	HIGH	Microsecond timestamp (%f) added to filename.
4	OCR returning empty/garbage strings	**RESOLVED**	HIGH	Plate-strip crop + 3x upscale + dual threshold + conf filter.
5	Repeat offender branch dropping first event	**RESOLVED**	MEDIUM	New event row always appended regardless of repeat status.
6	Night-time detection degraded	**OPEN**	MEDIUM	IR camera support planned for future version.
7	winsound limits portability to Windows	**OPEN**	LOW	Replace with playsound for cross-platform audio in v2.
8	VAHAN DB lookup not implemented	**OPEN**	LOW	Planned as Future Scope – requires MoRTH API access.